Please see pages 23-29 for the LLM's answers and my annotations on the trace.

mistral-chat

U

Please follow the instructions in this notebook. You will implement a Transformer using fundamental building
blocks in PyTorch. You'll apply the Transformer encoder-decoder model to a sequence-to-sequence
NLP task: document summarization. Refer to the Attention is All You Need paper for details on the model
architecture.
Once you finished with the notebook, answer the following questions in your submission of the written
assignment:
(a) Please submit the screenshots of the training loss and the validation loss displayed on Tensorboard.

M

Thank you for uploading the notebook file. I will review the contents of `q_summarize_part_1.py` to understand the instructions and the Transformer implementation task. After that, I'll guide you through implementing the Transformer model for document summarization and help you generate the required training and validation loss plots using TensorBoard.

Let's start by opening and reviewing the notebook file.

U

-*- coding: utf-8 -*-
"""q_summarize_part_1.ipynb
Automatically generated by Colab.
Original file is located at
https://colab.research.google.com/github/Berkeley-CS182/cs182fa25_public/blob/main/hw10/code/summarize/q_summarize_part_1.ipynb
Transformer for Summarization (Part I)
In this coding assignment, you'll implement a Transformer using

Please see pages 23-29 for the LLM's answers and my annotations on the trace.

fundamental building blocks in PyTorch. You'll apply the Transformer encoder-decoder model to a sequence-to-sequence NLP task: document summarization. Refer to the "Attention is All You Need" paper (https://arxiv.org/abs/1706.03762) for details on the model architecture. Any differences between our implementation and the paper will be noted throughout this notebook.

Note: Ensure you run this notebook on a CUDA GPU to optimize training speed. For instance, you can use a GPU instance on Google Colab.

Note: This notebook uses the tensorboard magic in Google Colab. If you're working in a different environment, you might need to find alternative solutions.

"""

```
# Commented out IPython magic to ensure Python compatibility.
#@title Install Packages
!pip install datasets==3.6.0
!pip install evaluate transformers rouge-score nltk
!pip install --no-deps --upgrade IPython
# %load_ext autoreload
# %autoreload 2
#@title Import Packages
import base64
import functools
import gzip
import json
import time
import einops
import torch
from datasets import load_dataset
from torch import nn
from torch.nn import functional as F
from transformers import AutoTokenizer, DataCollatorForSeq2Seq,
Seq2SeqTrainer, Seq2SeqTrainingArguments
from transformers.modeling_outputs import Seq2SeqLMOutput
TO_SAVE = {"time": time.time()}
```

"""## Data Processing

We'll use the **XSum** dataset (https://arxiv.org/abs/1808.08745) to train and evaluate our Transformer model. Each example in the XSum

Please see pages 23-29 for the LLM's answers and my annotations on the trace.

dataset consists of a document and its corresponding summary. Below are the dataset statistics and a sample example:
"""
xsum = load_dataset("xsum", trust_remote_code=True, download_mode="force_redownload")
xsum
xsum['train'][0]
"""Due to the original XSum dataset's large size, we'll use a subset for training and evaluation that primarily contains science and technology-related news and is about 10% as large as the original dataset. The following code cells will extract this subset for you."""
#@title Binary Blob of Subset Indices
blob =
b'H4sIAIHOLWQC/zyd24EFqQpFU5kA5uP4ADSWm38eV9aqno8+u8vy
rZQKCP/73xj//jPi33/m/Pefvd5f/vtP/t7fC8/3f+33V//+cx6eh4ME88UYe3fS
2T/9ovi57+e+t/M3+6dzHqt/dv9U/5wuraPMfrE6rEue+6Wd8cqc2f/V6J9
+e/rxdn73pV2/92JNfl7ktfq/9d6u/eufjtK1WrH6p6N0Q1a3ZJ1+2/Xbncv+
7f55b3dXcncNdrw679NhHXl34dHxousc3U8x+3G9FNF9EN1nEf22y43
osHhpo/rn9IvTkc/LObrwuKN/qrt6dV/T4e9tztPd3i+6D7J7vX6//uG/F6Xm
e1Hda7Vn//Qg9YhVrh6q0T8MWsfrMk5ncLqg0xU/s8O69idP/9we2ZfL6
Vqd+17crtDtkb7dxbe7+HYX393/dRffbu/tpt7Dz6va7bbd+/4bvy5u/Ab/d6
nvt/g9/Rs9iX63J8vokXy/HX8s/l8dZ2z+7+YNZ9noVo7RjXu/nfPsOr3fDpn
kObsT3i9vDyE9imP1dHzZd8zVPTgWpSwm8erGDObK++16riJ+EecQfkh
1iXk7ZA+poMM3OWzS7lr8Br/EhDZ2z4QRO/jtnIMSI6FBUgW1jetv55y/Ts
vseL+EUP+k1bk6h+yZMDL9JX7Pg/fLW8rN4/9QaU/+UavDi36upsDXqby
lD6uJ5v0eSLtzrh7j90s4dTtNy+MwCoeaHHr17M7n0LoThATxGcHDCB5
qdRijQ6+eu/l4dPxLzpdaXcbo0m9Mu3Fp1yUtk+79Tn4J70n/fvur8fvxO5I
vEp8kvjrMw/nrer5f4gThTf3vl/Dkfz5Bv8Nbvmyje+/98kXrj8cc5M8cfh86
foOY4f/ETP4vwvm2MYff57BrMi6fxuu3sXObg08kn8xJzZnn02/lpOaT+iz
auli/+Ngyz98v4XxqmfMvg+S3S1lNL+/r23VYZ/LLW2qyqMnuL8f75f/F/z1
b3i8f7eT/3PwSh3Yx/ydzfu6eM++3/w/6LahtjMlvx4wZ/HYOfDxn0EvRo/x
+SXsIJ7dgPYjLSkErMv0l5PA/LcqeUe83WU5YVGhFLdaWpppZjHj1LJ1Fz
tVfsMlX8/12Dof+PKRlnk/m+TzU9jBbTtPI++0WHWbLJS2f0bd68T853O
0vIYzIZQZeWs3cfgsdy52Lnqtef1Hf2sf/rHm/Ff3bY7GYt4t5u37J2x6X92
vl7d/O//3y9vCWnEd/W97E6Zij+2SN7tXF9/b9Eoe0o1v3fns5HZdU3bdvir
Ead3sXM3YxYxff4TUPlZd1uufSYsa+hduVm/+7Z94vCzotWkFMlu5FHdY
hPjVnli7W7/fbOWx2BJue2bSCWbp28Ja2MGPfb+fJXF18pRdf6cWMfb/E
b8p6ywK/9A8r//tlU7H87ZjBriPlM+jnoJ58yRdzdYW5XVL1nFx82xer//vtE

EJgyKKfG+tSYnrJl/Dmn5cAW7UAOmJQT0WJPpXlsqdZ97mq5okVlc2B
AhwUs0YNKvWuJxPY5sH+yDcpZqzH9z3UtN3a//3BXy3b4yKArN7BoZy
U6r8WWPle0nKMatUu1rmQxc4C+EqGnVOsstVQaqT+HtN5F9aok7FBa
kVlcqVSjSlsdHD7PwfMsqVQy2pZsCCULmEPklexgJV+JRZHq2MGsN1L
dKK2zLIlUIkb8ubH3f93YKIt5S0oESksilYTFS26/0uy05CfBja3aUgoy+2/n
tIgFcvuVtkp+zmk7pFNHZIy1JVcbJP4c1Y9c1HJXK33Vt7JBVxQ0h8MrrZ
sc8q5k+yq59/5c1HJaK2W79GSnbc0eW4KeLQ9eS3yvZQdwILsWwfdf7
ufH+4VkMf5cznItv/5/y9ks97OcsrLitvq01hhtiaKJVKK/uldPpa07rV6uJUf
QGqlba6LWIp4WLdhB6lpewBYdzeHpWpbe1mjeIke2PByt0bxFQMdpjXN
aufzL/Sw3uUbz1kadVq/Yqr0t627L5dCaZ1p+qOVraI3yrX6y1UP29sYpuX
i1raLVT7aCYjhkXcve26rtLTJQq7a3anvLztCyM7SoPy3bgsPR/bmrdVwp
HLuf9VQa2R1orlXDW/7BVq9oIaFWDW/ZFj5ntp5WWyxaa67WfLXVZ7Z
kBVoWhg7vBJRTWfW8tbZqWRVaPr5W3e6030COAwW0cHi5Vp1vWRJ
adjOHlbNoUGv+acGgVph1iwa1NrC1hC8dPq61kmpZDFobe/9c3TqrN8L
hrTfC1a3nFCmzNcojGvTn6ravE6fGz05V6379Ob6lKvazuN7PWl0/S3P9
ubPtEfbetZ8dYVbz+a+P225iXN1WqiQQ3M/xYP6349uuYLs0fpaitYLPuB
7tg3FilqO1es+AbrdG5c+SXERyI4Dbz3qUP8tW/6xW/bPovTV8xkXuVMp
Xlj3evs/hlH5W1SLo2p8b3A+4uaF9A651XOT2+nrb2s8atP/1jfuSg1P8X2
5wJ22xWWvvjBvcfmy7piw4+XOciJ+9GkRH+1kk9udYij9LSv7s7LDQzkO
8NMKl/RylxeI549fyDYGP2+f8LNarI2zaz9GGCYr28742S+cM2El9fNAv5i
1uP6u//yyW9XPMxJ+ll3/e5/bTHPe/nmsnbXXXn8XfrY1zwRsuHfEUd/bP8
Zl+lsv6WfXuZ5Esa+OMq/u1q9u/7KT2Ds/HDBCr3YyrO/7p+E47xfGb4/8r
31c+2L5EPj1ipf25yB/A5177v52RG9e3MZQ9oZozz/V2MXpzqOO0WPZ
mgIP2httt/ecptxt8//7pKd++5PigH/61I9It7rESJZtFx6H4n//8P3Rb7x7/Ag
MA'
train_indices, validation_indices, test_indices =
json.loads(gzip.decompress(base64.b64decode(blob)).decode("utf-
8"))
dataset_train = xsum["train"].select(train_indices)
dataset_validation = xsum["validation"].select(validation_indices)
dataset_test = xsum["test"].select(test_indices)
dataset_train, dataset_validation, dataset_test
"""### Tokenization and Collating
Before feeding the data into the model, we must convert raw texts into sequences of indices. Since our focus isn't on NLP specifics, we'll skip tokenization details and directly reuse the tokenizer and vocabulary from a pre-trained T5 model. The `tokenize` function tokenizes both the document and summary of each example before adding them to the

data structure.
"""
tokenizer = AutoTokenizer.from_pretrained("t5-small")
def tokenize(tokenizer, examples):
inputs = [doc for doc in examples["document"]]
model_inputs = tokenizer(inputs, max_length=1024, truncation=True)
labels = tokenizer(examples["summary"], max_length=128, truncation=True)
model_inputs["labels"] = labels["input_ids"]
return model_inputs
tokenize_fn = functools.partial(tokenize, tokenizer)
dataset_train = dataset_train.map(tokenize_fn, batched=True)
dataset_validation = dataset_validation.map(tokenize_fn, batched=True)
dataset_test = dataset_test.map(tokenize_fn, batched=True)
dataset_train[0].keys()
"""During training, tokenized examples are collated into batches before being fed into the model. We'll use `DataCollatorForSeq2Seq` from the `transformers` library, which conveniently handles padding and tensor conversions of both inputs and outputs for us."""
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model="t5-small")
"""## Implement Transformer
In this section, you'll implement the Transformer step by step. Starting with scaled dot-product attention and multi-head attention, you'll progress to Transformer layers and ultimately the Transformer encoder-decoder model. Please refer to the "Attention is All You Need" paper for implementation specifics.
Scaled Dot-Product Attention
In this section, you'll implement Scaled Dot-Product Attention. The input types and shapes are:
- `q`: `Tensor[n, tgt_len, d_head]`
- `k`: `Tensor[n, src_len, d_head]`
- `v`: `Tensor[n, src_len, d_head]`
- `key_padding_mask`: `Tensor[n, src_len]`
- `causal`: `bool`
`n` represents the total number of attention operations calculated in parallel. In multi-head attention, it's usually the product of the batch

size and the number of attention heads. For each of the `n` operations, compute attention scores
$$s_{i,j} = \mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_\text{head}}$$
Apply softmax to the attention score, then use it as weights to linearly combine values in `v`:
$$a_{i,j} = \dfrac{\exp(s_{i,j})}{\sum_k \exp(s_{i, k})}$$
$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_i = \sum_j a_{i,j} \mathbf{v}_j$$
However, consider two essential details: *padding* and *causal masking*.
- *padding*: The Transformer input usually contains sentences of varying lengths, padded into a tensor. Attention should ignore pad tokens, so they don't impact the results. `key_padding_mask` is a byte tensor set to **1** in pad token positions. If `key_padding_mask` is `None`, there's no padding in the input.
- *causal masking*: Autoregressive generation in the decoder uses causal attention masks, meaning position i can only attend to position j if $i \ge j$. If `causal` is set to true, apply causal attention masking. The provided `future_mask` may be useful.
Implement scaled dot-product attention below. Ensure your implementation is fully vectorized (no for-loops allowed). Avoid using `torch.nn.functional.multi_head_attention_forward` or any classes in `torch.nn`.
Hint: Consider using einops (https://einops.rocks/).
Hint: Alternatively, `torch.bmm` or `torch.einsum` may be helpful.
Hint: To mask an element in the attention map, set its pre-attention score to `-inf`, ensuring its post-attention score is always 0.
"""
future_mask = torch.triu(torch.zeros([1024, 1024]).fill_(float("-inf")), 1)
future_mask
def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
##
TODO: implement this function
##
##

```
######################
"""Ensure your code passes the following tests:"""
def test_scaled_dot_product_attention():
q1 = torch.tensor([[1, 0, 0], [0.5, 0.5, 0]]).view(1, 2, 3).float()
k1 = torch.tensor([[1, 0, 0], [0, 1, 0], [0, 0, 1]]).view(1, 3, 3).float()
v1 = torch.tensor([[3, 0, 0], [0, 5, 0], [0, 0, 7]]).view(1, 3, 3).float()
o1 = scaled_dot_product_attention(q1, k1, v1)
assert list(o1.shape) == [1, 2, 3]
assert torch.allclose(
o1.view(-1)[:5],
torch.tensor([1.413249135017395, 1.3222922086715698,
1.8512091636657715, 1.0912044048309326,
1.818674087524414]).float(),
rtol=1e-3
)
torch.manual_seed(100)
q2 = torch.randn(3, 5, 7).float()
k2 = torch.randn(3, 11, 7).float()
v2 = torch.randn(3, 11, 7).float()
o2 = scaled_dot_product_attention(q2, k2, v2)
assert list(o2.shape) == [3, 5, 7]
assert torch.allclose(
o2.view(-1)[6: 11],
torch.tensor([-0.40304261445999146, -0.2931785583496094,
0.20563912391662598, 0.08719107508659363,
0.08274038136005402]).float(),
rtol=1e-3
)
torch.manual_seed(200)
q3 = torch.randn(7, 5, 6).float()
k3 = torch.randn(7, 3, 6).float()
v3 = torch.randn(7, 3, 6).float()
o3 = scaled_dot_product_attention(q3, k3, v3)
TO_SAVE["scaled_dot_product_attention.3.shape"] = list(o3.shape)
TO_SAVE["scaled_dot_product_attention.3.value"] = o3.view(-1)[5:
10].tolist()
key_padding_mask = torch.tensor([[0, 0, 1]]).byte()
o4 = scaled_dot_product_attention(q1, k1, v1,
```

```
key_padding_mask=key_padding_mask)
assert list(o4.shape) == [1, 2, 3]
assert torch.allclose(
o4.view(-1)[:5],
torch.tensor([1.921372413635254, 1.7977124452590942, 0.0, 1.5,
2.5]),
rtol=1e-3
)
torch.manual_seed(210)
q5 = torch.randn(2, 4, 3).float()
k5 = torch.randn(2, 4, 3).float()
v5 = torch.randn(2, 4, 3).float()
o5 = scaled_dot_product_attention(q5, k5, v5, causal=True)
assert list(o5.shape) == [2, 4, 3]
assert torch.allclose(
o5.view(-1)[2: 7],
torch.tensor([0.9079901576042175, -0.573272705078125,
-1.1765587329864502, 0.7771514058113098,
-0.3235766291618347]),
rtol=1e-3
)
torch.manual_seed(220)
q6 = torch.randn(3, 5, 4).float()
k6 = torch.randn(3, 5, 4).float()
v6 = torch.randn(3, 5, 4).float()
key_padding_mask = torch.tensor([
[0, 0, 0, 0, 1],
[0, 0, 0, 0, 0],
[0, 0, 0, 1, 1]
]).byte()
o6 = scaled_dot_product_attention(q6, k6, v6,
key_padding_mask=key_padding_mask, causal=True)
TO_SAVE["scaled_dot_product_attention.6.shape"] = list(o6.shape)
TO_SAVE["scaled_dot_product_attention.6.value"] = o6.view(-1)[3:
8].tolist()
test_scaled_dot_product_attention()
"""### Multi-head Attention
In this section, you'll implement multi-head attention.
```

The input to this layer has types and shapes:
- `q`: `Tensor[bsz, tgt_len, d_model]`
- `k`: `Tensor[bsz, src_len, d_model]`
- `v`: `Tensor[bsz, src_len, d_model]`
- `key_padding_mask`: `Tensor[bsz, src_len]`
- `causal`: `bool`

A multi-head attention layer has four linear projection layers (including biases in this assignment): `q_proj`, `k_proj`, `v_proj`, and `o_proj`. `q_proj`, `k_proj`, and `v_proj` project `q`, `k`, and `v` respectively into `n_heads` `d_head` vectors. The shapes of the projected query, key, and value will be `[bsz, tgt_len, n_heads, d_head]`, `[bsz, src_len, n_heads, d_head]`, and `[bsz, src_len, n_heads, d_head]` respectively.

In the provided code below, instead of creating `n_heads` projection matrices of `d_model -> d_head` for the query/key/value, we use a single projection matrix of `d_model -> d_model`. This means the first `d_head` channels correspond to the first attention head, and channels from `d_head + 1` to `2 * d_head` correspond to the second attention head, and so on. The same rule applies to the input channels of `o_proj`.

Next, rearrange the projected query, key, and value tensors appropriately and feed them into your previously implemented `scaled_dot_product_attention` function.

The output of `scaled_dot_product_attention` should then be projected by o_proj to produce the final output. The output shape should be `[bsz, tgt_len, d_model]`.

Implement multi-head attention below. Avoid using `nn.MultiheadAttention` or `torch.nn.functional.multi_head_attention_forward`. Ensure your implementation is fully vectorized.

"""

```
class MultiheadAttention(nn.Module):
def __init__(self, d_model, n_heads):
super().__init__()
self.q_proj = nn.Linear(d_model, d_model)
self.k_proj = nn.Linear(d_model, d_model)
self.v_proj = nn.Linear(d_model, d_model)
self.o_proj = nn.Linear(d_model, d_model)
```

```
self.n_heads = n_heads
self.d_head = d_model // n_heads
def forward(self, q, k, v, key_padding_mask=None, causal=False):
############################################################
####################
# TODO: implement this method
############################################################
####################
############################################################
####################
"""The following code checks your implementation against
`nn.MultiheadAttention` in PyTorch:"""
def test_multihead_attention():
torch.manual_seed(350)
mha0 = nn.MultiheadAttention(embed_dim=128, num_heads=4,
batch_first=True)
nn.init.normal_(mha0.in_proj_weight, mean=0.0, std=0.05)
nn.init.normal_(mha0.in_proj_bias, mean=0.0, std=0.05)
nn.init.normal_(mha0.out_proj.weight, mean=0.0, std=0.05)
nn.init.normal_(mha0.out_proj.bias, mean=0.0, std=0.05)
mha1 = MultiheadAttention(128, 4)
mha1.q_proj.weight.data.copy_(mha0.in_proj_weight.data[:128, :])
mha1.q_proj.bias.data.copy_(mha0.in_proj_bias.data[:128])
mha1.k_proj.weight.data.copy_(mha0.in_proj_weight.data[128:256, :])
mha1.k_proj.bias.data.copy_(mha0.in_proj_bias.data[128:256])
mha1.v_proj.weight.data.copy_(mha0.in_proj_weight.data[256:, :])
mha1.v_proj.bias.data.copy_(mha0.in_proj_bias.data[256:])
mha1.o_proj.weight.data.copy_(mha0.out_proj.weight.data)
mha1.o_proj.bias.data.copy_(mha0.out_proj.bias.data)
torch.manual_seed(400)
q1 = torch.randn(4, 6, 128).float()
k1 = torch.randn(4, 6, 128).float()
v1 = torch.randn(4, 6, 128).float()
assert torch.allclose(
mha0(q1, k1, v1)[0].contiguous(),
mha1(q1, k1, v1).contiguous(),
rtol=1e-3
)
```

```
torch.manual_seed(500)
q2 = torch.randn(2, 5, 128).float()
k2 = torch.randn(2, 3, 128).float()
v2 = torch.randn(2, 3, 128).float()
o20 = mha0(q2, k2, v2)[0].contiguous()
o21 = mha1(q2, k2, v2).contiguous()
TO_SAVE["multihead_attention.2.0.shape"] = list(o20.shape)
TO_SAVE["multihead_attention.2.0.value"] = o20.view(-1)[126:
131].tolist()
TO_SAVE["multihead_attention.2.1.shape"] = list(o21.shape)
TO_SAVE["multihead_attention.2.1.value"] = o21.view(-1)[126:
131].tolist()
torch.manual_seed(600)
q3 = torch.randn(4, 6, 128).float()
k3 = torch.randn(4, 6, 128).float()
v3 = torch.randn(4, 6, 128).float()
key_padding_mask = torch.tensor([
[0, 0, 1, 1, 1, 1],
[0, 0, 0, 0, 1, 1],
[0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 1]
]).byte()
o30 = mha0(
q3, k3, v3,
key_padding_mask=key_padding_mask.to(torch.bool),
attn_mask=future_mask[:6, :6]
)[0].contiguous()
o31 = mha1(q3, k3, v3, key_padding_mask=key_padding_mask,
causal=True).contiguous()
assert torch.allclose(o30[0, :2], o31[0, :2], rtol=1e-3)
assert torch.allclose(o30[0, :4], o31[0, :4], rtol=1e-3)
assert torch.allclose(o30[0, :6], o31[0, :6], rtol=1e-3)
assert torch.allclose(o30[0, :5], o31[0, :5], rtol=1e-3)
torch.manual_seed(700)
q4 = torch.randn(2, 5, 128).float()
k4 = torch.randn(2, 5, 128).float()
v4 = torch.randn(2, 5, 128).float()
key_padding_mask = torch.tensor([
```

```
[0, 0, 1, 1, 1],
[0, 0, 0, 0, 1],
]).byte()
o40 = mha0(
q4, k4, v4,
key_padding_mask=key_padding_mask.to(torch.bool),
attn_mask=future_mask[:5, :5]
)[0].contiguous()
o41 = mha1(q4, k4, v4, key_padding_mask=key_padding_mask,
causal=True).contiguous()
TO_SAVE["multihead_attention.4.0.shape"] = list(o40.shape)
TO_SAVE["multihead_attention.4.0.value"] = o40.view(-1)[126:
131].tolist()
TO_SAVE["multihead_attention.4.1.shape"] = list(o41.shape)
TO_SAVE["multihead_attention.4.1.value"] = o41.view(-1)[126:
131].tolist()
test_multihead_attention()
"""### Transformer Layers
In this section, you'll **implement Transformer Encoder/Decoder
layers** according to Figure 1 of "Attention is All You Need". Note that
you should also apply residual dropout (Section 5.4 of the paper).
Activation dropout and attention dropout will not be used in this
assignment.
The `is_decoder` flag determines if this Transformer layer is an
encoder or a decoder.
If it's an encoder, the input types and shapes will be:
- `x`: `Tensor[bsz, src_len, d_model]`
- `padding_mask`: `Tensor[bsz, src_len]`
If it's a decoder, the input types and shapes will be:
- `x`: `Tensor[bsz, tgt_len, d_model]`
- `padding_mask`: `Tensor[bsz, tgt_len]`
- `encoder_out`: `Tensor[bsz, src_len, d_model]`
- `encoder_padding_mask`: `Tensor[bsz, src_len]`
The output should be a tensor of the same shape as `x`.
Avoid using `nn.TransformerEncoderLayer` or
`nn.TransformerDecoderLayer`.
"""
class TransformerLayer(nn.Module):
```

```
def __init__(self, is_decoder, d_model, n_heads, d_ffn, p_drop):
super().__init__()
self.is_decoder = is_decoder
self.self_attn = MultiheadAttention(d_model, n_heads)
self.self_attn_drop = nn.Dropout(p_drop)
self.self_attn_ln = nn.LayerNorm(d_model)
if is_decoder:
self.cross_attn = MultiheadAttention(d_model, n_heads)
self.cross_attn_drop = nn.Dropout(p_drop)
self.cross_attn_ln = nn.LayerNorm(d_model)
self.fc1 = nn.Linear(d_model, d_ffn)
self.fc2 = nn.Linear(d_ffn, d_model)
self.ffn_drop = nn.Dropout(p_drop)
self.ffn_ln = nn.LayerNorm(d_model)
def forward(self, x, padding_mask, encoder_out=None,
encoder_padding_mask=None):
##########################################################
####################
# TODO: implement this method
##########################################################
####################
##########################################################
####################
"""The following code checks your implementation against
`nn.TransformerEncoderLayer` and `nn.TransformerDecoderLayer` in
PyTorch:"""
def test_transformer_layer():
torch.manual_seed(750)
enc_layer0 = nn.TransformerEncoderLayer(128, 4,
dim_feedforward=512, dropout=0.0, batch_first=True)
nn.init.normal_(enc_layer0.self_attn.in_proj_weight, mean=0.0,
std=0.05)
nn.init.normal_(enc_layer0.self_attn.in_proj_bias, mean=0.0, std=0.05)
nn.init.normal_(enc_layer0.self_attn.out_proj.weight, mean=0.0,
std=0.05)
nn.init.normal_(enc_layer0.self_attn.out_proj.bias, mean=0.0, std=0.05)
nn.init.normal_(enc_layer0.linear1.weight, mean=0.0, std=0.05)
nn.init.normal_(enc_layer0.linear1.bias, mean=0.0, std=0.05)
```

```
nn.init.normal_(enc_layer0.linear2.weight, mean=0.0, std=0.05)
nn.init.normal_(enc_layer0.linear2.bias, mean=0.0, std=0.05)
enc_layer1 = TransformerLayer(False, 128, 4, 512, 0.0)
enc_layer1.self_attn.q_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[:128, :])
enc_layer1.self_attn.q_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[:128])
enc_layer1.self_attn.k_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[128:256, :])
enc_layer1.self_attn.k_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[128:256])
enc_layer1.self_attn.v_proj.weight.data.copy_(enc_layer0.self_attn.in_proj_weight.data[256:, :])
enc_layer1.self_attn.v_proj.bias.data.copy_(enc_layer0.self_attn.in_proj_bias.data[256:])
enc_layer1.self_attn.o_proj.weight.data.copy_(enc_layer0.self_attn.out_proj.weight.data)
enc_layer1.self_attn.o_proj.bias.data.copy_(enc_layer0.self_attn.out_proj.bias.data)
enc_layer1.fc1.weight.data.copy_(enc_layer0.linear1.weight.data)
enc_layer1.fc1.bias.data.copy_(enc_layer0.linear1.bias.data)
enc_layer1.fc2.weight.data.copy_(enc_layer0.linear2.weight.data)
enc_layer1.fc2.bias.data.copy_(enc_layer0.linear2.bias.data)
torch.manual_seed(800)
x = torch.randn(4, 5, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0, 0], [0, 0, 0, 1, 1], [0, 0, 0, 0, 1], [0, 0, 1, 1, 1]]).byte()
y10 = enc_layer0(x,
src_key_padding_mask=x_mask.to(torch.bool)).contiguous()
y11 = enc_layer1(x, x_mask).contiguous()
assert torch.allclose(y10[0], y11[0], rtol=1e-3)
assert torch.allclose(y10[1, :3], y11[1, :3], rtol=1e-3)
assert torch.allclose(y10[2, :4], y11[2, :4], rtol=1e-3)
assert torch.allclose(y10[3, :2], y11[3, :2], rtol=1e-3)
torch.manual_seed(900)
x = torch.randn(3, 4, 128).float()
x_mask = torch.tensor([[0, 0, 0, 1], [0, 0, 0, 1], [0, 0, 1, 1]]).byte()
y20 = enc_layer0(x,
```

```
src_key_padding_mask=x_mask.to(torch.bool)).contiguous()
y21 = enc_layer1(x, x_mask).contiguous()
TO_SAVE["transformer_layer.2.0.shape"] = list(y20.shape)
TO_SAVE["transformer_layer.2.0.value"] = y21.view(-1)[126: 131].tolist()
TO_SAVE["transformer_layer.2.1.shape"] = list(y20.shape)
TO_SAVE["transformer_layer.2.1.value"] = y21.view(-1)[126: 131].tolist()
torch.manual_seed(950)
dec_layer0 = nn.TransformerDecoderLayer(128, 4,
dim_feedforward=512, dropout=0.0, batch_first=True)
nn.init.normal_(dec_layer0.self_attn.in_proj_weight, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.self_attn.in_proj_bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.self_attn.out_proj.weight, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.self_attn.out_proj.bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.in_proj_weight, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.in_proj_bias, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.out_proj.weight, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.multihead_attn.out_proj.bias, mean=0.0,
std=0.05)
nn.init.normal_(dec_layer0.linear1.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear1.bias, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear2.weight, mean=0.0, std=0.05)
nn.init.normal_(dec_layer0.linear2.bias, mean=0.0, std=0.05)
dec_layer1 = TransformerLayer(True, 128, 4, 512, 0.0)
dec_layer1.self_attn.q_proj.weight.data.copy_(dec_layer0.self_attn.in_p
roj_weight.data[:128, :])
dec_layer1.self_attn.q_proj.bias.data.copy_(dec_layer0.self_attn.in_pro
j_bias.data[:128])
dec_layer1.self_attn.k_proj.weight.data.copy_(dec_layer0.self_attn.in_p
roj_weight.data[128:256, :])
dec_layer1.self_attn.k_proj.bias.data.copy_(dec_layer0.self_attn.in_proj
_bias.data[128:256])
dec_layer1.self_attn.v_proj.weight.data.copy_(dec_layer0.self_attn.in_p
roj_weight.data[256:, :])
```

```
dec_layer1.self_attn.v_proj.bias.data.copy_(dec_layer0.self_attn.in_proj_bias.data[256:])
dec_layer1.self_attn.o_proj.weight.data.copy_(dec_layer0.self_attn.out_proj.weight.data)
dec_layer1.self_attn.o_proj.bias.data.copy_(dec_layer0.self_attn.out_proj.bias.data)
dec_layer1.cross_attn.q_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[:128, :])
dec_layer1.cross_attn.q_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[:128])
dec_layer1.cross_attn.k_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[128:256, :])
dec_layer1.cross_attn.k_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[128:256])
dec_layer1.cross_attn.v_proj.weight.data.copy_(dec_layer0.multihead_attn.in_proj_weight.data[256:, :])
dec_layer1.cross_attn.v_proj.bias.data.copy_(dec_layer0.multihead_attn.in_proj_bias.data[256:])
dec_layer1.cross_attn.o_proj.weight.data.copy_(dec_layer0.multihead_attn.out_proj.weight.data)
dec_layer1.cross_attn.o_proj.bias.data.copy_(dec_layer0.multihead_attn.out_proj.bias.data)
dec_layer1.fc1.weight.data.copy_(dec_layer0.linear1.weight.data)
dec_layer1.fc1.bias.data.copy_(dec_layer0.linear1.bias.data)
dec_layer1.fc2.weight.data.copy_(dec_layer0.linear2.weight.data)
dec_layer1.fc2.bias.data.copy_(dec_layer0.linear2.bias.data)
torch.manual_seed(1000)
x = torch.randn(4, 5, 128).float()
e = torch.randn(4, 3, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0, 0], [0, 0, 0, 1, 1], [0, 0, 0, 0, 1], [0, 0, 1, 1, 1]]).byte()
e_mask = torch.tensor([[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 1]]).byte()
y30 = dec_layer0(x, e, tgt_mask=future_mask[:5, :5],
tgt_key_padding_mask=x_mask.to(torch.bool),
memory_key_padding_mask=e_mask.to(torch.bool)).contiguous()
y31 = dec_layer1(x, x_mask, e, e_mask).contiguous()
assert torch.allclose(y30[0], y31[0], rtol=1e-3)
assert torch.allclose(y30[1, :3], y31[1, :3], rtol=1e-3)
```

```
assert torch.allclose(y30[2, :4], y31[2, :4], rtol=1e-3)
assert torch.allclose(y30[3, :2], y31[3, :2], rtol=1e-3)
torch.manual_seed(1100)
x = torch.randn(3, 4, 128).float()
e = torch.randn(3, 3, 128).float()
x_mask = torch.tensor([[0, 0, 0, 0], [0, 0, 0, 1], [0, 0, 1, 1]]).byte()
e_mask = torch.tensor([[0, 0, 0], [0, 0, 0], [0, 0, 1]]).byte()
y40 = dec_layer0(x, e, tgt_mask=future_mask[:4, :4],
tgt_key_padding_mask=x_mask.to(torch.bool),
memory_key_padding_mask=e_mask.to(torch.bool)).contiguous()
y41 = dec_layer1(x, x_mask, e, e_mask).contiguous()
TO_SAVE["transformer_layer.4.0.shape"] = list(y40.shape)
TO_SAVE["transformer_layer.4.0.value"] = y41.view(-1)[126: 131].tolist()
TO_SAVE["transformer_layer.4.1.shape"] = list(y40.shape)
TO_SAVE["transformer_layer.4.1.value"] = y41.view(-1)[126: 131].tolist()
test_transformer_layer()
"""### Putting them together: Transformer
In this section, you will implement a Transformer encoder-decoder
model for sequence-to-sequence tasks using the building blocks
you've already created: scaled dot-product attention, multi-head
attention, and Transformer encoder/decoder layers.
Model Overview:
- Encoder: `n_layers` layers
- Decoder: `n_layers` layers
- Learned positional embeddings instead of sinusoidal positional
encodings (as in "Attention is All You Need")
- Shared embedding matrices to reduced number of parameters and to
improve training stability:
- Encoder input embeddings
- Decoder input embeddings
- Decoder output layer weights (a linear classifier over n_words
vocabulary, whose weight matrix happens to have the same shape as
word embeddings, so their weights can be shared)
- Layer normalization after the embedding layers
- Shared positional embedding matrix and normalization layer for both
encoder and decoder
- Decoder's first input token: `[EOS]`
- Handling of pad tokens in labels (-100). Huggingface `transformers`
```

pads labels with padding index -100 instead of `pad_id`. So we do processing as follows:
- Replace -100 in decoder input
- Ignore -100 in labels when calculating loss
Implement the `make_positions` method to generate input for the positional embedding layer. The output of `make_positions` should be a tensor with the same dtype and shape as `input_ids`. For example, if the input is a batch of 2 sequences with a length of 5, the output of `make_positions` should be:
```
[[0, 1, 2, 3, 4],
[0, 1, 2, 3, 4]]
```
"""
class Transformer(nn.Module):
def __init__(self, n_words, pad_id, eos_id, max_len, n_layers, d_model, n_heads, d_ffn, p_drop):
super().__init__()
self.pad_id = pad_id
self.eos_id = eos_id
self.emb_word = nn.Embedding(n_words, d_model)
self.emb_pos = nn.Embedding(max_len, d_model)
self.emb_word.weight.data.uniform_(-0.05, 0.05)
self.emb_pos.weight.data.uniform_(-0.05, 0.05)
self.emb_ln = nn.LayerNorm(d_model)
self.encoder_layers = nn.ModuleList([
TransformerLayer(False, d_model, n_heads, d_ffn, p_drop)
for _ in range(n_layers)
])
self.decoder_layers = nn.ModuleList([
TransformerLayer(True, d_model, n_heads, d_ffn, p_drop)
for _ in range(n_layers)
])
self.lm_head = nn.Linear(d_model, n_words)
self.lm_head.weight = self.emb_word.weight
self.criterion = nn.CrossEntropyLoss(ignore_index=-100)
def make_positions(self, input_ids, padding_mask):
##

```
####################
# TODO: implement this method
##################################################################
####################
##################################################################
####################
def forward(self, input_ids, attention_mask, labels):
enc_padding_mask = input_ids.eq(self.pad_id).byte()
enc_pos = self.make_positions(input_ids, enc_padding_mask)
enc_state = self.emb_ln(self.emb_word(input_ids) +
self.emb_pos(enc_pos))
for layer in self.encoder_layers:
enc_state = layer(enc_state, enc_padding_mask)
decoder_input_ids = labels.new_zeros(labels.shape)
decoder_input_ids[:, 1:] = labels[:, :-1].clone()
decoder_input_ids[:, 0] = self.eos_id
decoder_input_ids.masked_fill_(decoder_input_ids == -100,
self.pad_id)
dec_padding_mask = decoder_input_ids.eq(self.pad_id).byte()
dec_pos = self.make_positions(decoder_input_ids,
dec_padding_mask)
dec_state = self.emb_ln(self.emb_word(decoder_input_ids) +
self.emb_pos(dec_pos))
for layer in self.decoder_layers:
dec_state = layer(dec_state, dec_padding_mask, enc_state,
enc_padding_mask)
lm_logits = self.lm_head(dec_state)
loss = self.criterion(lm_logits.view(-1, lm_logits.size(-1)),
labels.view(-1))
return Seq2SeqLMOutput(
loss=loss,
logits=lm_logits
)
"""## Train the Transformer for Summarization
Once the model and the data are both ready, we can begin training our
Transformer encoder-decoder model for summarization. In the
following code cell, we define a Transformer encoder-decoder model
with a 4-layer encoder and a 4-layer decoder, both containing 8
```

attention heads with a dimension of 64.
"""
model = Transformer(
tokenizer.vocab_size,
tokenizer.pad_token_id,
tokenizer.eos_token_id,
max_len=1024,
n_layers=4,
d_model=512,
n_heads=8,
d_ffn=2048,
p_drop=0.1
)
"""We'll start a Tensorboard to visualize the training of the model.
Note: When started in Google Colab, Tensorboard does not default to automatically update, so please click the 🔄 symbol on the top-right to manually update the Tensorboard. Or open the `Settings -> Reload data`.

⚠️ The following code is a workaround (2023 October), due to some recent update in Google Chrome.
The issue is currently live:
<https://github.com/googlecolab/colabtools/issues/3990>.
Hopefully it would not be long before it is fixed.
"""
# Commented out IPython magic to ensure Python compatibility.
# %load_ext tensorboard
# ignore the error in the output and run the next cell
# %tensorboard --logdir my_xsum_model/logs --host=127.0.0.1 --port=6006 --load_fast=false
from google.colab import output
output.serve_kernel_port_as_window(6006, path="")
# click the link https://localhost:6006
"""We'll train the model using the following hyperparameters with `Seq2SeqTrainer` from Huggingface's `transformers`. The trainer will automatically move the model to the GPU, set it to training mode, and run forward pass, backward pass, and optimization steps for us. It will also update the Tensorboard you started above (press the refresh

button at the top right corner of the Tensorboard page to reload it if it does not update).
Training will take a considerable amount of time (~20 minutes on a Tesla T4 GPU), so you're not required to tune these hyperparameters. Just **ensure that your implementation is correct**. In this case, the validation loss of your model after 2 epochs should be close to **5.0**. The training runs in FP16 mixed precision mode, which generally speeds up training, especially on newer GPUs (like the ones available if you're using Colab Pro). This is a common practice in modern NLP. However, if your implementation has numerical stability issues, it could cause instability during training. If you encounter precision-related issues, double-check your implementation of scaled dot-product attention, and *ensure that softmax is applied to FP32 tensors* (since softmax in FP16 mode loses too much precision).
Question
Please report your final validation accuracy after 2 epochs in your written assignment, along with screenshots of the training loss and the validation loss displayed on Tensorboard.
"""

```
import os
os.environ["WANDB_DISABLED"] = "true"
training_args = Seq2SeqTrainingArguments(
output_dir="my_xsum_model",
learning_rate=1e-4,
weight_decay=0.01,
warmup_ratio=0.1,
per_device_train_batch_size=16,
gradient_accumulation_steps=1,
dataloader_drop_last=True,
per_device_eval_batch_size=16,
num_train_epochs=2,
predict_with_generate=False,
push_to_hub=False,
logging_dir="my_xsum_model/logs",
logging_strategy="steps",
logging_first_step=True,
logging_steps=1,
save_strategy="epoch",
```

```
eval_strategy="epoch",
fp16=True,
)
trainer = Seq2SeqTrainer(
model=model,
args=training_args,
train_dataset=dataset_train,
eval_dataset=dataset_validation,
tokenizer=tokenizer,
data_collator=data_collator,
)
trainer.train()
"""## Submission
You are done with this coding assignment. Please download
`submission_log.json` and submit it to Gradescope.
You might be thinking: wait, why hasn't the model been tested on real evaluation metrics like ROUGE score? Why haven't we looked at any examples of the model's generated text yet? That's because this is Part I of the assignment.
The Transformer model you implemented above cannot be used for efficient autoregressive generation. For each token that needs to be decoded, it needs to run a forward pass of the encoder for the document and a forward pass of the decoder for all tokens in the generated summary. This would be disastrously slow even for a single example.
In the following weeks, you will work on Part II, which will cover efficient sequence generation using a Transformer by caching computed encoder/decoder states. Moreover, you will learn to fine-tune pretrained language models for significantly better performance.
"""
with open("submission_logs.json", "w", encoding="utf-8") as f:
json.dump(TO_SAVE, f)
```

M

I'll help you implement the missing parts of your Transformer notebook step by step, focusing on the core components: **scaled dot-product attention**, **multi-head attention**, **Transformer layers**, and the **Transformer model**. I'll also guide you on how to generate the required screenshots for your assignment.

1. Scaled Dot-Product Attention

This function computes the attention scores, applies masking (padding and causal), and returns the weighted sum of values.

python
Copy

```
def scaled_dot_product_attention(q, k, v, key_padding_mask=None,
causal=False):
    # q: [n, tgt_len, d_head]
    # k: [n, src_len, d_head]
    # v: [n, src_len, d_head]
    # key_padding_mask: [n, src_len]
    # causal: bool

    # Compute attention scores
    d_head = q.size(-1)
    scores = torch.bmm(q, k.transpose(1, 2)) / (d_head ** 0.5)  #
[n, tgt_len, src_len]

    # Apply causal mask if needed
    if causal:
        mask = future_mask[:scores.size(1), :scores.size(2)].to(s
cores.device)
        scores += mask

    # Apply key padding mask if needed
    if key_padding_mask is not None:
        scores.masked_fill_(key_padding_mask.unsqueeze(1), float
("-inf"))

    # Softmax
    attn_weights = F.softmax(scores, dim=-1)

    # Weighted sum of values
    output = torch.bmm(attn_weights, v)  # [n, tgt_len, d_head]

    return output
```

Mostly Correct
Missing FP32 softmax cast (will cause training failure)

---

2. Multi-Head Attention

Mostly Correct
Doesn't handle different seq lengths (cross-attention will fail)

This class implements multi-head attention using the `scaled_dot_product_attention` function.

python

Copy

```python
class MultiheadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.o_proj = nn.Linear(d_model, d_model)
        self.n_heads = n_heads
        self.d_head = d_model // n_heads

    def forward(self, q, k, v, key_padding_mask=None, causal=Fals
e):
        # q, k, v: [bsz, seq_len, d_model]
        bsz, seq_len, _ = q.size()

        # Project and reshape
        q = self.q_proj(q).view(bsz, seq_len, self.n_heads, self.
d_head).transpose(1, 2)  # [bsz, n_heads, seq_len, d_head]
        k = self.k_proj(k).view(bsz, seq_len, self.n_heads, self.
d_head).transpose(1, 2)
        v = self.v_proj(v).view(bsz, seq_len, self.n_heads, self.
d_head).transpose(1, 2)

        # Reshape for scaled_dot_product_attention
        q = q.contiguous().view(-1, seq_len, self.d_head)  # [bsz
* n_heads, seq_len, d_head]
        k = k.contiguous().view(-1, seq_len, self.d_head)
        v = v.contiguous().view(-1, seq_len, self.d_head)

        # Apply attention
        if key_padding_mask is not None:
            key_padding_mask = key_padding_mask.unsqueeze(1).expa
nd(-1, self.n_heads, -1).contiguous().view(-1, seq_len)
        output = scaled_dot_product_attention(q, k, v, key_paddin
g_mask, causal)

        # Reshape and project
        output = output.view(bsz, self.n_heads, seq_len, self.d_h
ead).transpose(1, 2).contiguous().view(bsz, seq_len, -1)
        output = self.o_proj(output)

        return output
```

---

3. Transformer Layer

Incorrect

Wrong normalization order (Pre-LN vs Post-LN), wrong activation (GELU vs ReLU)

This class implements a single Transformer layer (encoder or decoder).

python

Copy

```
class TransformerLayer(nn.Module):
    def __init__(self, is_decoder, d_model, n_heads, d_ffn, p_dro
p):
        super().__init__()
        self.is_decoder = is_decoder
        self.self_attn = MultiheadAttention(d_model, n_heads)
        self.self_attn_drop = nn.Dropout(p_drop)
        self.self_attn_ln = nn.LayerNorm(d_model)
        if is_decoder:
            self.cross_attn = MultiheadAttention(d_model, n_head
s)
            self.cross_attn_drop = nn.Dropout(p_drop)
            self.cross_attn_ln = nn.LayerNorm(d_model)
        self.fc1 = nn.Linear(d_model, d_ffn)
        self.fc2 = nn.Linear(d_ffn, d_model)
        self.ffn_drop = nn.Dropout(p_drop)
        self.ffn_ln = nn.LayerNorm(d_model)

    def forward(self, x, padding_mask, encoder_out=None, encoder_
padding_mask=None):
        # Self-attention
        residual = x
        x = self.self_attn_ln(x)
        x = self.self_attn(x, x, x, padding_mask, causal=self.is_
decoder)
        x = self.self_attn_drop(x)
        x = x + residual

        # Cross-attention (decoder only)
        if self.is_decoder:
            residual = x
            x = self.cross_attn_ln(x)
            x = self.cross_attn(x, encoder_out, encoder_out, enco
der_padding_mask)
            x = self.cross_attn_drop(x)
            x = x + residual

        # Feed-forward network
        residual = x
        x = self.ffn_ln(x)
        x = self.fc2(F.gelu(self.fc1(x)))
        x = self.ffn_drop(x)
        x = x + residual
```

```
        return x
```

Incorrect
Wrong position calculation (padding tokens get position 0)

4. Transformer Model

This class implements the full Transformer encoder-decoder model.

python

Copy

```
class Transformer(nn.Module):
    def __init__(self, n_words, pad_id, eos_id, max_len, n_layer
s, d_model, n_heads, d_ffn, p_drop):
        super().__init__()
        self.pad_id = pad_id
        self.eos_id = eos_id
        self.emb_word = nn.Embedding(n_words, d_model)
        self.emb_pos = nn.Embedding(max_len, d_model)
        self.emb_word.weight.data.uniform_(-0.05, 0.05)
        self.emb_pos.weight.data.uniform_(-0.05, 0.05)
        self.emb_ln = nn.LayerNorm(d_model)
        self.encoder_layers = nn.ModuleList([
            TransformerLayer(False, d_model, n_heads, d_ffn, p_dr
op)
            for _ in range(n_layers)
        ])
        self.decoder_layers = nn.ModuleList([
            TransformerLayer(True, d_model, n_heads, d_ffn, p_dro
p)
            for _ in range(n_layers)
        ])
        self.lm_head = nn.Linear(d_model, n_words)
        self.lm_head.weight = self.emb_word.weight
        self.criterion = nn.CrossEntropyLoss(ignore_index=-100)

    def make_positions(self, input_ids, padding_mask):
        # input_ids: [bsz, seq_len]
        # padding_mask: [bsz, seq_len]
        bsz, seq_len = input_ids.size()
        positions = torch.arange(seq_len, device=input_ids.devic
e).expand(bsz, -1)
        positions.masked_fill_(padding_mask, 0)
        return positions

    def forward(self, input_ids, attention_mask, labels):
        enc_padding_mask = input_ids.eq(self.pad_id).byte()
        enc_pos = self.make_positions(input_ids, enc_padding_mas
```

```
k)
        enc_state = self.emb_ln(self.emb_word(input_ids) + self.e
mb_pos(enc_pos))
        for layer in self.encoder_layers:
            enc_state = layer(enc_state, enc_padding_mask)

        decoder_input_ids = labels.new_zeros(labels.shape)
        decoder_input_ids[:, 1:] = labels[:, :-1].clone()
        decoder_input_ids[:, 0] = self.eos_id
        decoder_input_ids.masked_fill_(decoder_input_ids == -100,
self.pad_id)
        dec_padding_mask = decoder_input_ids.eq(self.pad_id).byte
()
        dec_pos = self.make_positions(decoder_input_ids, dec_padd
ing_mask)
        dec_state = self.emb_ln(self.emb_word(decoder_input_ids)
+ self.emb_pos(dec_pos))
        for layer in self.decoder_layers:
            dec_state = layer(dec_state, dec_padding_mask, enc_st
ate, enc_padding_mask)

        lm_logits = self.lm_head(dec_state)
        loss = self.criterion(lm_logits.view(-1, lm_logits.size(-
1)), labels.view(-1))
        return Seq2SeqLMOutput(loss=loss, logits=lm_logits)
```

---

5. Training and TensorBoard Screenshots

To generate the required screenshots for your assignment:

1. **Run the training cell** in your notebook. This will start the training process and log metrics to TensorBoard.
2. **Open TensorBoard** by clicking the link provided in the notebook (e.g., `https://localhost:6006`).
3. **Take screenshots** of the following:
    - **Training Loss**: Navigate to the "Scalars" tab and select the "loss" metric.
    - **Validation Loss**: Navigate to the "Scalars" tab and select the "eval_loss" metric.

---

6. Validation Accuracy

After 2 epochs, your validation loss should be close to **5.0**. If your implementation is correct, you should see this value in the TensorBoard logs or the training output.

---

7. Submission

After training, download the `submission_logs.json` file and submit it to Gradescope. This file contains the logs and metrics required for your assignment.